Shomer.AI — Manual Test Flows (Android phone / emulator + live server)

Use this when: you sit down with the Android client (physical phone **or** emulator) and a locally-running FastAPI server, and want to walk the whole monitoring loop by hand and tick it off. **Companion (authoritative, scriptable):** `integration/integration-monitor.md`. **Classifier under test:** trained **DictaBERT D10** (`CLASSIFIER_MODEL_VERSION=v1.1-dictabert`, macro-F1 0.836) — this is the *real* model now, not the old Ollama stand-in.

Print this, fill the **Result / PASS-FAIL** column as you go, note the date + device at the top.

```
Tester: _____________   Date: 2026-06-09   Device: ☐ physical phone  ☐ emulator   Server port: _____
```

---

0. One-time setup (do once before the flows)

#	Step	Command / action	Done
0.1	Start the server from the **repo root** (`--host 0.0.0.0` so a phone can reach it; fresh port avoids orphaned `:8000`)	`cd C:\AIDevelopmentCourse\Shomer.AI` `$env:DIGEST_ALLOW_MANUAL_TRIGGER="true"` `.\server\.venv\Scripts\python.exe -m uvicorn server.app.main:app --host 0.0.0.0 --port 8011`	☐
0.2	Confirm the model loaded	In the startup log, look for the DictaBERT checkpoint load (no Ollama fallback warning). `irm http://localhost:8011/v1/model/info` → version `v1.1-dictabert`.	☐
0.3	Find the PC's LAN IP (physical phone only)	`ipconfig` → IPv4 Address, e.g. `192.168.1.40`	☐
0.4	Open the firewall once (physical phone only)	`New-NetFirewallRule -DisplayName "OffensiveHebrew" -Direction Inbound -LocalPort 8011 -Protocol TCP -Action Allow`	☐
0.5	Build + install the **`client`** flavor (uninstall any old `poc` APK first — different applicationId)	Android Studio → run `client` flavor, **or** `cd android_client; .\gradlew :app:installClientDebug`	☐
0.6	Set the app's base URL	Emulator → `http://10.0.2.2:8011` · Physical phone → `http://<PC-LAN-IP>:8011`	☐

> **Real alerts vs. silent:** `violence` always escalates to the Context Agent first. With **no LLM key** the CA is a mock that resolves to "not a threat" → **silent** (no alert). To see a real violence alert, put a `GEMINI_API_KEY` (or OpenAI/Anthropic) in `server/.env` and keep `CONTEXT_AGENT_ENABLED=true`. `abusive` / `hate` (high-confidence) and `pornographic` fire **direct** alerts without the CA.

---

1. Provision a parent + pairing code (server side)

Run on the PC; keep the three tokens for the rest of the flows.

```
$base = "http://localhost:8011"
$p  = irm "$base/v1/parent/register" -Method Post -ContentType application/json -Body '{"display_name"
```

```
$ph = @{ Authorization = "Bearer $($p.parent_token)" }
$c  = irm "$base/v1/parent/children" -Method Post -Headers $ph -ContentType application/json -Body '{"
$code = (irm "$base/v1/parent/pairing-code" -Method Post -Headers $ph -ContentType application/json -B
"parent_token=$($p.parent_token)`nchild_id=$($c.child_id)`npairing_code=$code"
```

Check	Expected	Result
`register` returns a `parent_token`	non-empty opaque token	
`children` returns a `child_id`	non-empty	
`pairing-code` returns a short `code` (OTP)	6-ish chars, time-limited	

Record: `parent_token = _____` `child_id = _____` `code = _____`

---

2. FLOW A — Onboarding & pairing (child mode, on the device)

#	Step	Expected	Result
A1	Launch the app, pick **Child** role	Role chooser → consent screen	
A2	Read + accept **Consent**	Screen discloses **both inbound and outbound** message capture; Accept enabled only after scroll/consent	
A3	**Pairing**: enter the OTP `code` from §1	App reports paired; shows the `child_id` matching §1	
A4	Watch the server log	A line like `identity.pair_endpoint_success` for that child	

PASS = app is paired and displays the same `child_id` the server issued.

> Behind the scenes A3 calls `POST /v1/pair` (OTP → opaque **device token**). The app stores that token and uses it as the `Bearer` for every upload — no parent token ever lives on the child device.

---

3. FLOW B — Permissions & the monitoring indicator

#	Step	Expected	Result
B1	Grant the **AccessibilityService** (app deep-links to Settings → Shomer → enable)	Toggle on; back in app it shows "monitoring active"	
B2	Grant **POST_NOTIFICATIONS** (Android 13+)	Permission dialog accepted	
B3	Grant **battery exemption** (so WorkManager uploads aren't deferred)	Allowed	
B4	Confirm the **non-dismissible** "monitoring active" notification	Persistent notification present; cannot be swiped away	

PASS = accessibility on + persistent indicator visible. (This is the consent/transparency guarantee.)

---

4. FLOW C — Capture an offensive Hebrew message → flag

Use a target app the service can read (WhatsApp recommended). Send/receive these one at a time and watch the server. **Tip:** keep `irm "$base/v1/parent/alerts" -Headers $ph` handy in a second PowerShell window.

#	Send/receive in WhatsApp (Hebrew)	Meaning	Expected server outcome	Result
C1	`אתה מטומטם ואני שונא אותך`	"you're an idiot, I hate you" — **abusive**	`monitor.batch_received` → `monitor.flagged`; flag label ≈ `abusive`, **direct alert**	
C2	`אני אהרוג אותך מחר`	"I'll kill you tomorrow" — **violence**	flagged + **escalates to Context Agent**. Alert only if an LLM key is set (else silent — by design)	
C3	`מה נשמע? נתראה מחר בבית הספר`	"what's up? see you at school" — **benign**	captured, classified `non_offensive`, **NOT flagged**	
C4	re-send the **exact** C1 text again	duplicate	**deduped** (no new flag) — `accepted` but `deduped=1`	

Check	Expected	Result
App counters increment	captured / uploaded counters go up after the upload interval	
C1 produces a flag	appears in `/v1/parent/alerts` with `quote` matching, `app_package=com.whatsapp`	
C3 does **not** produce a flag	benign text stays unflagged (low false-positive behavior)	
C4 dedups	no second flag for identical text	

PASS = offensive captured + flagged, benign not flagged, duplicate deduped.

> Uploads are batched by WorkManager, so allow up to the upload interval (not instant). If nothing arrives, see Troubleshooting (network reachability is the usual cause).

---

5. FLOW D — Flag appears server-side & in the daily digest

```
irm "$base/v1/parent/alerts" -Headers $ph                # the C1 flag should be here
irm "$base/internal/digest/run" -Method Post             # force the once-a-day digest now
$today = Get-Date -Format "yyyy-MM-dd"
irm "$base/v1/parent/digests/$today" -Headers $ph
```

Check	Expected	Result
Alert list contains the C1 capture	`quote`, `label`, `severity`, `direction` populated	
`digest/run` returns `digests_built` ≥ 1	digest built for today	
Digest body	lists `total_flagged`, `review_needed`, `by_severity`, `by_label`	

PASS = flag is queryable and rolls up into today's digest.

---

6. FLOW E — Parent review & react (web dashboard)

#	Step	Expected	Result
E1	Open `dashboard/index.html` (double-click or serve via the app's StaticFiles)	Hebrew RTL parent surface loads	
E2	Settings → Base URL `http://localhost:8011` + paste the `parent_token` from §1	Saved to localStorage; alert list populates	
E3	The C1 flag is in the list	shows quote + label + severity	
E4	Open it → **acknowledge**	status flips to acknowledged on re-list	
E5	**Label** it (e.g. `offensive`)	`parent_label` persists	
E6	**Change severity**	new severity persists	
E7	Confirm the human label is exportable	`irm "$base/v1/parent/labels/export" -Headers $ph` includes the labeled item	

PASS = each reaction (ack / label / severity) persists across a refresh and the label exports for training.

---

7. FLOW F — Parent review on the Android app (parent mode)

#	Step	Expected	Result
F1	On a second device/emulator (or after re-pairing as parent), pick **Parent** role	Parent auth screen	
F2	Register or paste the `parent_token` from §1	Authenticated; alert list loads	
F3	Alert list shows the C1 flag	quote + label visible	
F4	Open detail → react (ack / label / severity)	persists (poll refresh shows new status)	
F5	Open the digest screen	today's digest renders	

PASS = parent mode shows the same flags the dashboard does and reactions persist server-side.

---

8. FLOW G — Security / negative checks (must hold)

#	Action	Expected	Result
G1	`POST /v1/monitor/events` with **no** `Authorization` header	**401**	
G2	`POST /v1/monitor/events` with a valid device token but a **different** `child_id` in the body	**403** (a child can't post for another child)	
G3	`GET /v1/parent/alerts` with **no** parent token	**401**	
G4	Inspect the wire (Studio Logcat / server log): does any raw monitored text leave the device except via the authenticated `/v1/monitor/events` call?	**No** other egress of raw text	

```
# G1 — no auth
try { irm "$base/v1/monitor/events" -Method Post -ContentType application/json -Body '{}' } catch { $_
# G2 — wrong child id (use the device token from a pairing + a bogus child_id)
```

PASS = `401` without a token, `403` cross-child, and no unauthenticated raw-text egress.

Sign-off

- [ ] **Flow A** — pairing works (device token issued, child_id matches)
- [ ] **Flow B** — permissions granted + persistent monitoring indicator
- [ ] **Flow C** — offensive captured & flagged, benign unflagged, duplicate deduped
- [ ] **Flow D** — flag queryable + in today's digest
- [ ] **Flow E** — dashboard react loop persists + label exports
- [ ] **Flow F** — parent-mode app shows flags + reacts (optional if only one device)
- [ ] **Flow G** — `401` / `403` / no-raw-egress all hold

```
Overall:  ☐ PASS   ☐ PASS WITH NOTES   ☐ FAIL        Notes: _____________________________________
```

Troubleshooting (the usual suspects)

Symptom	Likely cause	Fix
App can't reach server	wrong base URL / firewall	Emulator must use `10.0.2.2` (not `localhost`); phone uses the PC LAN IP + the firewall rule (§0.4); both must be on the same Wi-Fi
`CLEARTEXT communication not permitted`	HTTP (not HTTPS) on a release-ish config	dev `network_security_config.xml` must allow cleartext to the LAN host — it does in the `client` debug flavor
Pairing fails / code rejected	OTP expired	pairing codes are time-limited; mint a fresh one (§1)
Nothing captured from WhatsApp	AccessibilityService off or app killed	re-enable in Settings (§B1); grant battery exemption (§B3) so WorkManager runs
Captured but never uploaded	upload is batched	wait for the upload interval; check Logcat for `MonitorUploader`
Violence message didn't alert	CA mock with no LLM key	expected — set `GEMINI_API_KEY` + `CONTEXT_AGENT_ENABLED=true` for real violence alerts
Server log crashes on Hebrew	cp1252 console	`main.py` forces UTF-8; if you wrapped it, run via the venv python directly
Switching `poc` ↔ `client` flavor fails to install	different applicationId	uninstall the other flavor's APK first
Port `:8000` won't bind	orphaned background uvicorn	use a fresh port (8011) as in §0.1

Known limitations (by design at this stage — not failures)

- **Daily digest scheduler is manual** by default — forced via `POST /internal/digest/run` (`DIGEST_BACKEND=asyncio` + `DIGEST_HOUR` enables the real once-a-day cron).
- **FCM push is `LogNotifier`** — digest/alert delivery is *logged*, not pushed to a device, until a Firebase project + service-account JSON exist (`ALERTS_CHANNEL=fcm`). `ntfy` is a no-account alternative.
- **`MONITOR_STORE_RAW` enforcement is pending (S5)** — non-flagged raw text is still persisted server-side until the classify-and-discard blanking lands.
- **Direction inference is heuristic** — defaults to `inbound`; per-app accuracy improves in Android A5.
- **MediaProjection / ML-Kit OCR fallback** (for apps that block accessibility) is Android A5, not covered here.

- **Minority-class accuracy caveat** — porn/violence eval data was partly synthetic; in-the-wild labels may differ from the reported macro-F1.